Laborator 2
STRUCTURI FUNDAMENTALE DE PROGRAMARE

I. Obiective

3.1. Structuri alternative

3.2. Structuri repetitive

II. Consideraţii teoretice

2.1. STRUCTURI ALTERNATIVE

Structura **IF..THEN..END IF**

```
IF cond1 THEN
        Secventa instr1
ELSE
        Secventa instr2
END IF;

IF conditie1 THEN
        Secventa instr1
ELSE
IF conditie2 THEN
        Secventa instr2
END IF;
```

Se poate folosi clauza ELSIF în loc de IF imbricate

```
IF conditie1 THEN
        Secventa instr1
ELSIF  conditie2  THEN  Secventa  instr2;
ELSIF conditie3 THEN Secventa instr3; -
----------------------------------
ELSIF conditieN THEN Secventa instrn;
```

Exemplu:

În funcţie de valoarea stocului produsului cu codul 3133, acesta se va afişa modificat pe ecran prin intermediul variabilei v_stoc.

```
DECLARE
        v_stoc produse.stoc%type;
BEGIN
        SELECT stoc into v_stoc from produse where id_produs=3133;
        dbms_output.put_line ('Stocul initial este: '||v_stoc);
        IF v_stoc < 500 THEN
                v_stoc:=2* v_stoc;
        ELSIF v_stoc between 500 and 1000 THEN
                v_stoc:=1.5 * v_stoc;
        ELSE
                v_stoc:=1.25* v_stoc;
        END IF;
        dbms_output.put_line('Stocul final este: '||v_stoc);
END;
/
```

Atenţie la variabilele de tip NULL şi evaluarea în IF!

De exemplu, în următoarea situaţie se va afişa "Felicitări, sunteţi admis!" din cauza faptului că variabila nota este declarată, dar nu este iniţializată, fiind deci NULL:

```
DECLARE
        nota number;
BEGIN
        IF nota<5 THEN
                dbms_output.put_line('Ne pare rau, candidatul este respins!');
        ELSE
                dbms_output.put_line('Felicitari, sunteti admis!');
         END IF;
END;
/
```

Observaţi cazurile de mai jos:

```
X:=10;
Y:=NULL;
IF x!=y then
--intoarce NULL si nu TRUE
END IF;
```

sau

```
a:=NULL;
b:=NULL;
IF a=b then
--intoarce NULL si nu TRUE
END IF;
```

Structura **CASE ... WHEN... THEN...**

Sunt 2 variante:

1. **expresii CASE (CASE Expressions)** care întorc un rezultat într-o variabilă. Se termină cu END
2. **sintaxa CASE (CASE Statement)** care executa o anumita instructiune. Se termina cu END CASE, iar fiecare rand se termina cu ;

1. **CASE Expressions:**

```
Variabila:=
CASE [Selector]
WHEN expression1 THEN result1
WHEN expression2 THEN result2
----------------------------------------
WHEN expressionN THEN resultN
[ELSE result N+1]
END;
```

Exemplu:

```
DECLARE
        v_stoc produse.stoc%type;
        v_proc number(4,2);
BEGIN
        SELECT stoc
        INTO v_stoc
        FROM produse
        WHERE id_produs=3133;
        dbms_output.put_line('Stocul initial este: '||v_stoc);
        v_proc:=
                CASE
                        when v_stoc < 500 THEN 2
                        when v_stoc between 500 and 1000 THEN 1.5
                        else 1.25
                END;
        v_stoc:= v_stoc *v_proc;
        dbms_output.put_line('Salariul final este: '|| v_stoc);
END;
/
```

2. **CASE Statement:**

```
CASE [Selector]
WHEN expression1 THEN action1;
WHEN expression2 THEN action2;
----------------------------------------
WHEN expressionN THEN actionN;
[ELSE action N+1];
END CASE;
```

Exemplu:

```
DECLARE
        v_stoc produse.stoc%type;
BEGIN
        SELECT stoc
        INTO v_stoc
        FROM produse
        WHERE id_produs=3133;
        dbms_output.put_line('Stocul initial este: '||v_stoc);
        CASE
                when v_stoc < 500 THEN v_stoc:= v_stoc *2;
                when v_stoc between 500 and 1000 THEN v_stoc:= v_stoc *1.5;
                else v_stoc:= v_stoc *1.25;
        END CASE;
        dbms_output.put_line('Salariul final este: '|| v_stoc);
END;
/
```

2. 2. STRUCTURI REPETITIVE

Structura LOOP……END LOOP

```
LOOP
        Secventa comenzi;
        EXIT [WHEN cond];
END LOOP;
```

Exemplu:

Se afişează pe ecran utilizând structura loop…end loop numerele 9, 7, 4, 0.

```
set serveroutput on
DECLARE
        v_nr number(2):=10;
        i number(2):=1;
BEGIN
        loop
                v_nr:=v_nr-
                i; i:=i+1;
                exit when v_nr < 0;
                dbms_output.put_line(v_nr);
        end loop;
END;
/
```

Se afişează în ordine angajaţii cu codurile în intervalul 100 -110 atât timp cât salariul acestora este mai mic decât media:

```
DECLARE
        v_sal angajati.salariul%type;
        v_salMediu v_sal%type;
        i number(4):=100;
BEGIN
        SELECT avg(salariul)
        INTO v_salmediu
        FROM angajati;
        dbms_output.put_line('Salariul mediu este: '||v_salmediu);
        loop
           select salariul into v_sal from angajati where id_angajat=i;
           dbms_output.put_line('Salariatul cu codul '||i||' are salariul:'||v_sal);
           i:=i+1;
           exit when v_sal<v_salmediu or i>110;
        end loop;
END;
/
```

Structura WHILE…..LOOP….END LOOP

```
WHILE conditie LOOP
        Secventa comenzi 1;
        Secventa comenzi 2;
        EXIT [WHEN conditie];
END LOOP;
```

Exemplu:

Se afişează pe ecran utilizând structura while loop…end loop numerele 9,7, 4, 0.

```
set serveroutput on
DECLARE
        v_nr number(2):=10;
        i number(2):=1;
BEGIN
        while v_nr > 0 loop
                v_nr:=v_nr-
                i; i:=i+1;
                dbms_output.put_line(v_nr);
        end loop;
END;
/
```

Se afişează în ordine angajaţii cu codurile în intervalul 100 -110 atât timp cât salariul acestora este mai mic decât media:

```
DECLARE
        v_sal angajati.salariul%type;
        v_salMediu v_sal%type;
        i number(4):=100;
BEGIN
    SELECT avg(salariul)
    INTO v_salmediu FROM
    angajati;
    dbms_output.put_line('Salariul mediu este: '||v_salmediu);
    while i<=110 loop
            select salariul into v_sal from angajati where id_angajat=i;
            dbms_output.put_line('Salariatul cu codul '||i||' are salariul: '||v_sal);
            i:=i+1;
            exit when v_sal<v_salmediu;
     end loop;
END;
/
```

Structura FOR…..LOOP….END LOOP

```
FOR var IN [REVERSE] valmin..valmax LOOP
        Secventa comenzi;
        EXIT [WHEN conditie];
END LOOP;
```

Observatii:

Valorile intervalului pot fi de orice tip, dar să aibă valori care pot fi convertite la un întreg (de exemplu 20/13 sau 11/5). Dacă aceste 2 valori vor fi egale ca întregi atunci instrucțiunile din interiorul ciclului se execută o singură dată.

De exemplu secventa:

FOR i IN 3..3 LOOP
Secventa comenzi;
END LOOP;

Exemple:

Se afişează pe ecran utilizând structura for…end loop numerele 9,7, 4, 0.

```
set serveroutput on
DECLARE
        v_nr number(2):=10;
        i number(2);
BEGIN
        for i in 1..10 loop
                v_nr:=v_nr-i; exit
                when v_nr < 0;
                dbms_output.put_line(v_nr);
        end loop;
END;
/
```

Se afişează în ordine angajaţii cu codurile în intervalul 100 -110 atât timp cât salariul acestora este mai mic decât media:

```
DECLARE
        v_sal angajati.salariul%type;
        v_salMediu v_sal%type;
        -- i nu mai trebuie declarat
BEGIN
    SELECT avg(salariul) into v_salmediu from angajati;
    dbms_output.put_line('Salariul mediu este: '||v_salmediu);
    for i in 100..110 loop
        select salariul into v_sal from angajati where id_angajat=i;
          dbms_output.put_line('Salariatul cu codul '||i||' are salariul: '||v_sal);
          exit when v_sal<v_salmediu;
    end loop;
end;
/
```

Să se afişeze numărul de comenzi ale fiecărui angajat al cărui id este situat în intervalul 155..160, dar să se întrerupă afişarea în cazul în care se găseşte primul angajat din acest interval care nu are nici o comandă:

```
DECLARE
        v_nr number;
        v_nume angajati.nume%type;
        v_id angajati.id_angajat%type;
BEGIN
        for v_id in 155..160 loop
        v_nr:=0;
        SELECT count(c.nr_comanda)
        INTO v_nr
        FROM comenzi c,angajati a
        WHERE c.id_angajat=a.id_angajat and a.id_angajat=v_id;
        dbms_output.put_line('Salariatul cu id-ul: '||v_id||' are: '||v_nr||' comenzi');
        exit when v_nr=0;
  end loop;
END;
/
```

Să se încarce în tabela MESAJE numere de la 1…10 cu excepţia lui 6 şi 8.

```
CREATE TABLE MESAJE (REZ VARCHAR2(30));

BEGIN
FOR i IN 1..10 LOOP
   IF i=6 OR i=8 THEN
      NULL;
   ELSE
     INSERT INTO MESAJE(REZ) VALUES (i);
   END IF;
   COMMIT;
END LOOP;
END;
/
```

Structuri LOOP imbricate (se vor eticheta loop-urile)

```
BEGIN
        <<LOOP_EXTERN>>
        LOOP
                v_var:=v_var+1;
                EXIT WHEN v_var>10;
                <<LOOP_INTERN>>
                LOOP
                        ……….
                        EXIT LOOP_EXTERN WHEN
                        conditie1; EXIT WHEN conditie2;
                        ……….
                END LOOP;
                ……..
        END LOOP;
END;
```

III. Desfășurarea lucrării

Testare secvențelor în PL/SQL pe baza structurilor de baze de date create personal, utilizând exemplele prezentate.